



40. File System Implementation

Persistent Store

Similarity to Memory

Allocation Strategies

Many Different Approaches

Key Design Questions

1. Contiguous Allocation

2. Small # of Extents

3. Linked Allocation

4. File-Allocation Table(FAT)

5. Indexed Allocation

요약 비교표 - 확실하지는 않음

6. Multi-Level Indexing

Assume Multi-Level Indexing

On-Disk Structures

Data

Indexing Structures

Namespace

Allocation Metadata

Global Metadata

FS Structs: Empty Disk

Data Blocks: 56개 (8 ~ 63)

Inodes: 초록색은 inode block (index block or indirect block)

One Inode Block (초록색)

Inode & Inodes

예시

Allocation

Superblock

File Organization: The inode

File Structure: Indexed Allocation

Directories

Simple Directory List Example

Directory Structure

On-Disk Structures

Operations

create /foo/bar

open /foo/bar

write to /foo/bar

왜 inode를 두 번 접근할까?

read /foo/bar - assume opened

close /foo/bar

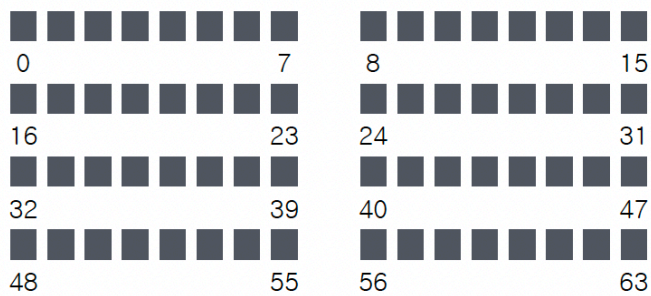
Efficiency

Write Buffering

Caching and Buffering

Summary

Persistent Store



- Given: large array of blocks on disk
- Want: some structure to map files to disk blocks

Similarity to Memory

Same principle: map logical abstraction to physical resource

Allocation Strategies

Many Different Approaches

- 6가지 방법론이 존재한다!!

1. **Contiguous**
2. **Extent-based**

- 3. **Linked**
- 4. **File-allocation Tables**
- 5. **Indexed**
- 6. **Multi-level Indexed** ← 지금 쓰는 거

Key Design Questions

Fragmentation (단편화)

- Internal & external fragmentation
내부 & 외부 단편화
- Free space that cannot be allocated effectively

File Growth

- Can the file grow dynamically without relocation?
파일이 커질 때 디스크에서 재배치하지 않고 확장할 수 있어야 해
- claude

문제가 되는 경우:

- **Contiguous Allocation (연속 할당):** 파일 뒤에 공간이 없으면 전체를 이동해야 함

잘 해결하는 경우:

- **Multi-level Indexing:** indirect block을 추가하면 동적으로 확장 가능
- **Linked Allocation:** 새 블록만 추가하면 돼

Sequential Access Performance (순차 접근 성능)

- How well does the layout support fast sequential reads/writes?
연속된 데이터를 읽을 때 얼마나 빠른가?
- claude

좋은 예:

- **Contiguous Allocation:** 디스크 헤드가 움직일 필요 없이 연속 읽기 가능 → 매우 빠름

나쁜 예:

- **Linked Allocation:** 각 블록이 디스크 곳곳에 흩어져 있어서 seek time이 많이 걸림

Random Access Efficiency (랜덤 접근 효율성)

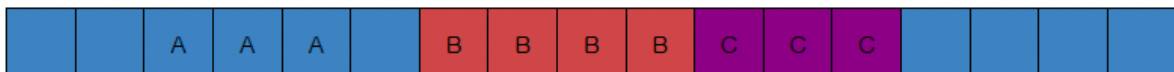
- How fast can the system locate arbitrary data blocks?
파일 중간의 임의 위치로 바로 접근할 수 있는가?

Metadata Overhead (메타데이터 오버헤드)

- How much space is consumed by metadata instead of data?
실제 데이터가 아닌, 파일을 관리하기 위한 정보가 얼마나 많은 공간을 차지하는가?
- Metadata must be stored **persistently**
- claude

할당 방식	메타데이터
contiguous	시작 위치 + 크기만 저장 → 작음
linked	각 블록마다 포인터 저장 → 중간
indexed	인덱스 블록 전체 필요 → 많음

1. Contiguous Allocation



- A: 12KB, B: 16KB,,,
- Allocate each file to contiguous sectors on disk
 - Meta-data: **Starting block and size of file**
Starting block (시작 블록 번호)
Size (파일 크기 또는 블록 개수)
 - OS allocates by finding sufficient free space
 - 필요한 크기만큼의 **연속된 빈 공간** 찾기
 - 공간이 있으면 할당, 없으면 실패
 - 미래 파일 크기를 예측해야 함; Should space be reserved?
 - claude

Option 1: 정확히 필요한 만큼만 할당
 → 파일 확장 시 이동 필요
 Option 2: 여유 공간 미리 예약
 → 공간 낭비 (internal fragmentation)

- Example: IBM OS/360
- **Fragmentation (internal and external)?**
 -) Horrible **external fragmentation** (needs periodic compaction)
 중간 중간에 비는 공간들이 생김
- **Ability to grow file over time?**
 -) May not be able to without moving
- **Seek cost for sequential accesses?**
 - + Excellent performance
 한 번의 seek으로 전체 읽기 가능!
- **Speed to calculate random accesses?**
 - + Simple calculation
- **Wasted space for meta-data?**
 - + Little overhead for meta-data

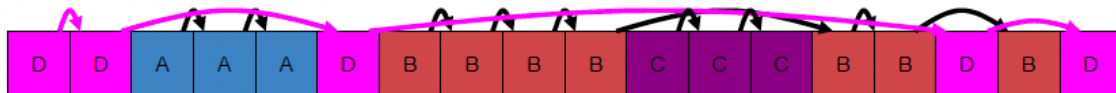
2. Small # of Extents



- Contiguous의 개선 버전
- 하나의 파일이 여러 개의 연속 구간으로 나뉨
- 각 extent는 내부적으로 contiguous
- extent 개수를 제한 (보통 2~6개)
- Allocate multiple contiguous regions (extents) per file
 각각의 extent는 contiguous, extent끼리는 contiguous할 필요는 없다, extent 단위로 할당한다

- Meta-data
 - Small array (2-6) designating each extent
 - Each entry: starting block and size
- Fragmentation (internal and external)?
 -) Helps external fragmentation: 기존의 외부 단편화를 어느정도 완화함
- Ability to grow file over time?
 -) Can grow (until run out of extents)
- Seek cost for sequential accesses?
 - +) Still Good performance
- Speed to calculate random accesses?
 - +) Still Simple calculation
- Wasted space for meta-data?
 - +) Still small overhead for meta-data

3. Linked Allocation



- Allocate linked-list of fixed-sized blocks (multiple sectors)
 - Meta-data: Location of first block of file, Each block also contains pointer to next block
첫번째 블록의 위치 및 각각의 블록이 포인터 갖고 있음
 - Examples: TOPS-10, Alto
- Fragmentation (internal and external)?
 - +) No external fragmentation (use any block)
internal? → 있을 수 있음 - block의 크기에 따라 다름
- Ability to grow file over time?
 - +) Can grow easily

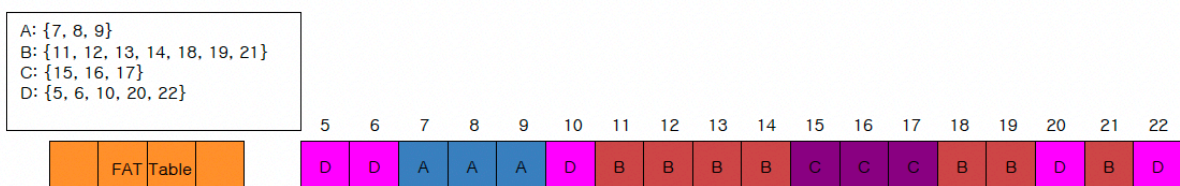
- Seek cost for sequential accesses?
 - + / -) Depends on data layout
- Speed to calculate random accesses?
 -) Ridiculously poor: 순차탐색을 하며 쫓아가야 해서
- Wasted space for meta-data?
 -) Waste pointer per block

Trade-off: Block size (does not need to equal sector size) - alignment가 안 맞는다?

한 줄 요약: 포인터 저장하느라 데이터 공간이 2의 제곱수(예: 512)가 아니라 애매한 숫자(예: 508)가 되어버려서

컴퓨터가 데이터를 읽고 쓸 때 **박자가 꼬이는 현상**

4. File-Allocation Table(FAT)

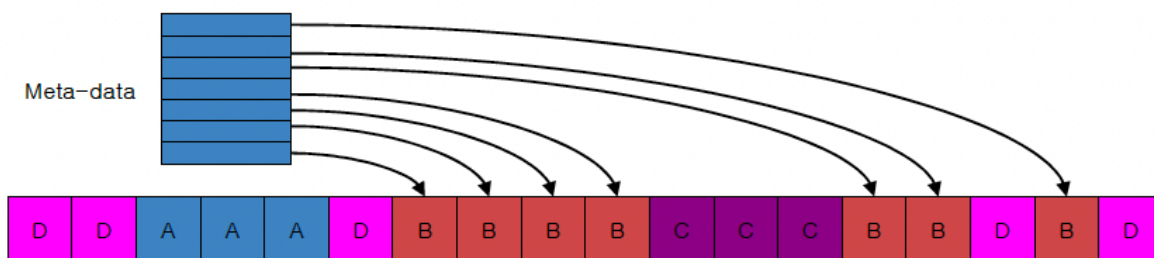


- Variation of Linked allocation
 - 포인터 대신에 meta table을 썼다?
 - Keep linked-list information for all files in on-disk FAT table
 - Meta-data: Location of first block of file
 - FAT table itself
- Draw corresponding FAT Table?
- Comparison to Linked Allocation
 - Same basic advantages and disadvantages
기본적인 장단점은 동일
 - Disadvantage: Read from two disk locations for every data read
단점: 매번 두 곳의 디스크를 읽어야 함 = 데이터를 읽을 때마다 두 번의 디스크 접근 필요

- FAT 테이블 읽기 (다음 블록 확인)
- 실제 데이터 블록 읽기
- Optimization: Cache FAT in main memory
FAT을 메인 메모리에 캐싱
 - Advantage: Greatly improves random accesses
장점: 랜덤 접근 성능이 크게 향상됨
FAT이 메모리에 있으면 디스크 접근 없이 다음 블록 위치를 바로 알 수 있음
 - What portions should be cached? Scale with larger file systems?
질문: 어느 부분을 캐싱해야 하나? 큰 파일 시스템에서는 어떻게 확장하나?

27:29까지 청취

5. Indexed Allocation



- 각 파일마다 전용 인덱스 블록을 가지는 방식
- Allocate fixed-sized blocks for each file
 - Meta-data: Fixed-sized array of block pointers
 - Allocate space for ptrs at file creation time
- Advantages
 - **No external fragmentation (외부 단편화 없음)**
 - Files can be easily grown up to max file size
왜? 빈 포인터 슬롯에 새 블록 추가만 하면 됨
 - Support random access
 - **비교: (CLAUDE)**
Contiguous: $O(1)$ - start + offset
Indexed: $O(1)$ - pointers[offset]

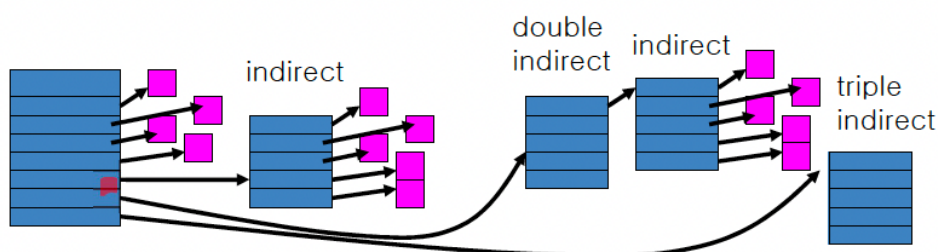
Linked: $O(N)$ - 체인 따라가기
 FAT: $O(N)$ - 체인 따라가기 (캐시 안 되면)

- Disadvantages
 - Large overhead for meta-data
 - Wastes space for unneeded pointers (most files are small!)
작은 파일이 대부분인데 공간낭비가 크다
 - data block이 있고 index block이 있음
 - block allocation unit은 4KB
 - 안 쓰는 공간(포인터 없는 곳)은 낭비
 - 파일 사이즈 MAX 정해져 있음?: 4MB → 이걸 해결하는게 multi-level indexing

요약 비교표 - 확실하지는 않음

항목	Contiguous	Linked	FAT	Indexed
External Frag	XXXX	✓	✓	✓
File Growth	XX	✓	✓	✓ (제한)
Random Access	✓✓✓	XXXX	✓(캐시)	✓✓✓
Metadata Overhead	✓✓✓	✓✓	✓✓	XXXX

6. Multi-Level Indexing



- single indirection, double indirection, triple indirection...
- Direct pointer: 실제 데이터 블록을 가리킴
- Indirect pointer: 인덱스 블록을 가리킴

- Variation of Indexed Allocation
 - Dynamically allocate hierarchy of pointers to blocks as needed
 - Meta-data: Small number of pointers allocated statically
 - Additional pointers to blocks of pointers
 - Examples: UNIX FFS-based file systems, ext2, ext3
- Comparison to Indexed Allocation
 - Advantage: Does not waste space for unneeded pointers
 - Still fast access for small files
 - Can grow to what size??
 - Disadvantage: Need to read indirect blocks of pointers to calculate addresses (extra disk read)
 - Keep indirect blocks cached in main memory

Assume Multi-Level Indexing

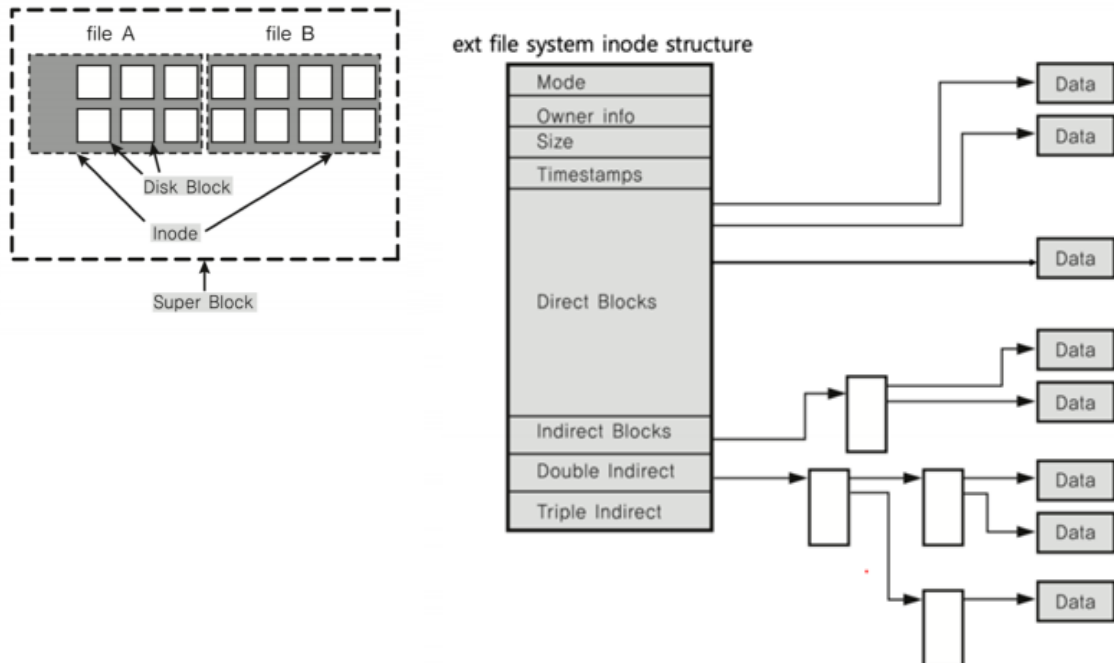
Why Multi-Level Indexing?

- Provides a simple and general abstraction for mapping file blocks
파일 블록 매핑을 위한 간단하고 범용적인 추상화를 제공함
- Scales to large files without excessive metadata
과도한 메타데이터 없이 대용량 파일까지 확장 가능
- Supports efficient random and sequential access
효율적인 랜덤 접근과 순차 접근을 모두 지원
- Forms the foundation of widely used file systems (e.g., UNIX FFS, ext2)
널리 사용되는 파일 시스템(예: UNIX FFS, ext2)의 기반을 형성

Key Idea

- Treat file data as a tree of block pointers
 - Direct blocks
 - Single indirect
 - Double indirect
 - Triple indirect

On-Disk Structures



- 작은 크기의 파일 많음 & 큰 크기의 파일 적음: 종류가 이렇게 구분된다? 대부분의 파일 들은?

Data

- **Data blocks**
 - File contents stored here

Indexing Structures

- **inode table - 아직 안 배움**
 - Per-file metadata + pointers to data blocks
파일별 메타데이터 + 데이터 블록을 가리키는 포인터들
- **Indirect blocks**
 - Extend file size beyond direct pointers
 - Direct 포인터만으로는 부족할 때 파일 크기 확장
 - 포인터의 포인터 역할

Namespace

- **Directories**
 - Map filenames → inode numbers
파일 이름을 inode 번호로 매핑

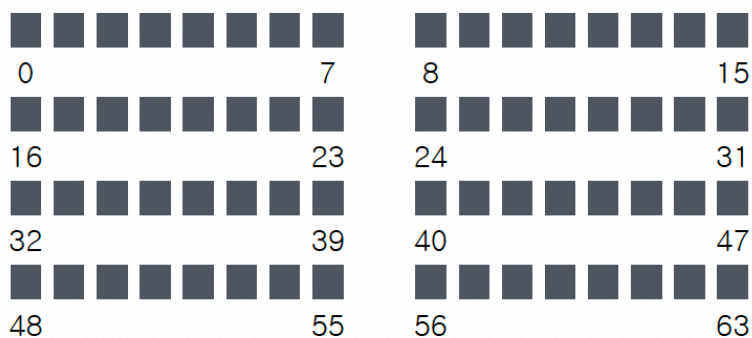
Allocation Metadata

- **Data bitmap**
 - Tracks free/used data blocks
- **Inode bitmap**
 - Tracks free/used inodes

Global Metadata

- **Superblock**
 - File system configuration, size, layout, status
 - Must be stored reliably (critical metadata)

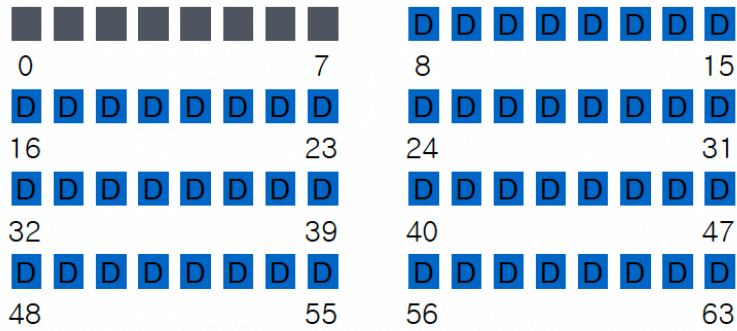
FS Structs: Empty Disk



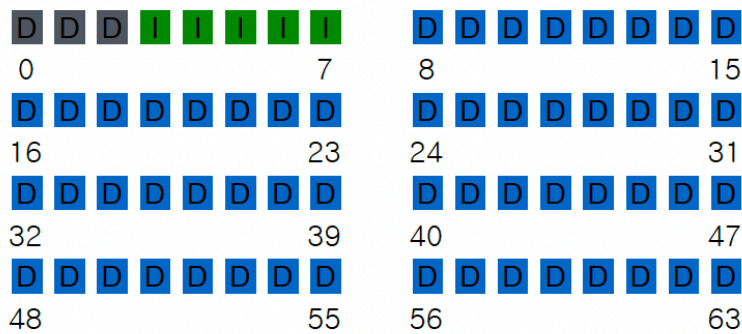
Assume each block is 4KB

- 64개

Data Blocks: 56개 (8 ~ 63)



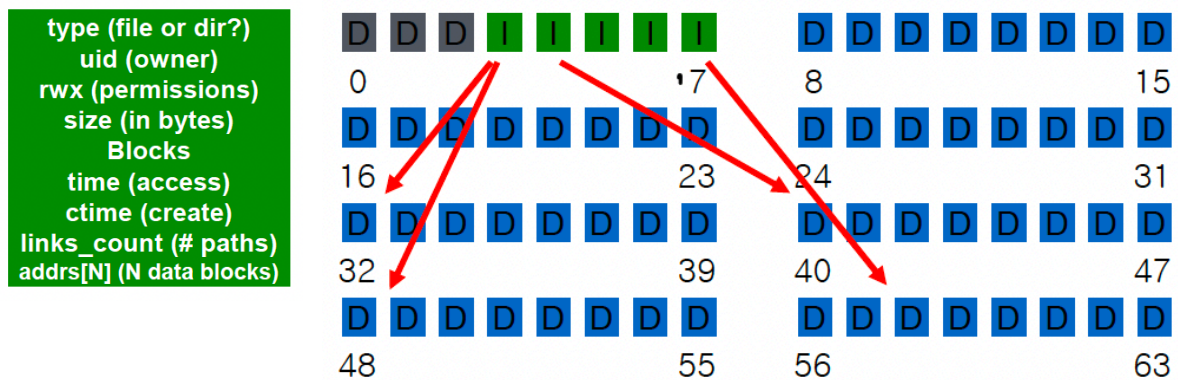
Inodes: 초록색은 inode block (index block or indirect block)



One Inode Block (초록색)

- 이 예시에서는 Indirect block이 없다고 가정
- Each inode is typically 256 bytes (depends on the FS, maybe 128 bytes)
- 4KB disk block
- 16 inodes per inode block
 - $4 * 2^{10} / 256 =$ 한 개의 inode block에 inode 16개가 들어간다
 - 그렇다면 최대 파일의 개수는?
 - $16 * 5 = 80$ 개 그러나 현재 56개가 최대임 이 가정에서는

Inode & Inodes



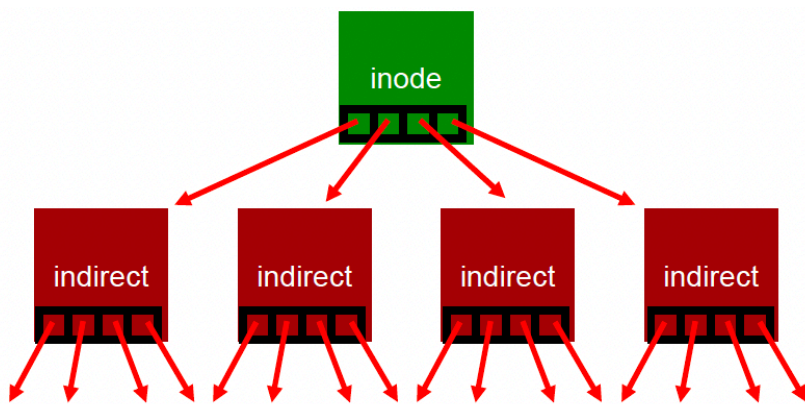
이 예시에서는 Indirect block이 없다고 가정

Assume single level (just pointers to data blocks)

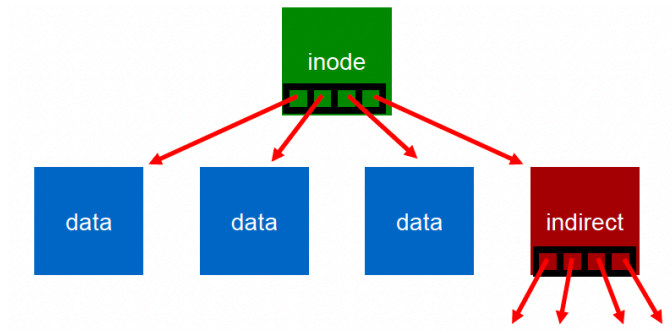
What is max file size?

- $256\text{byte} / 4\text{byte} = 64\text{개의 포인터}$
 $64 * 4\text{K} = 256\text{KB}$

How to get larger files?



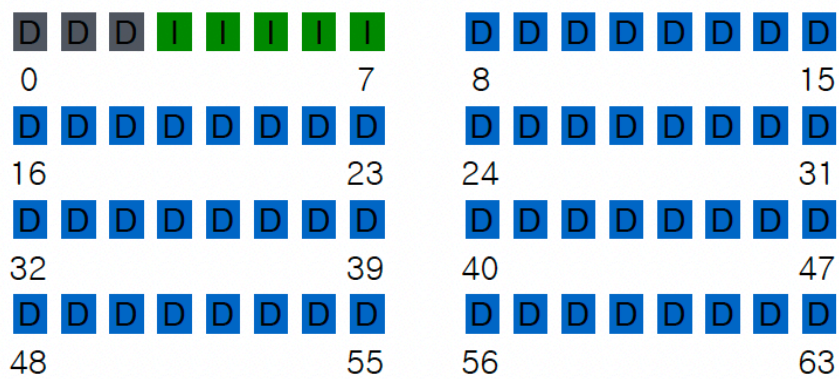
- Indirect blocks are stored in regular data blocks
- what if we want to optimize for small files?
 - 다음과 같이 해결



- 작은 건 direct하게

예시

- Assume 256 byte inodes (16 inodes/block).
- What is offset for inode with number 0?

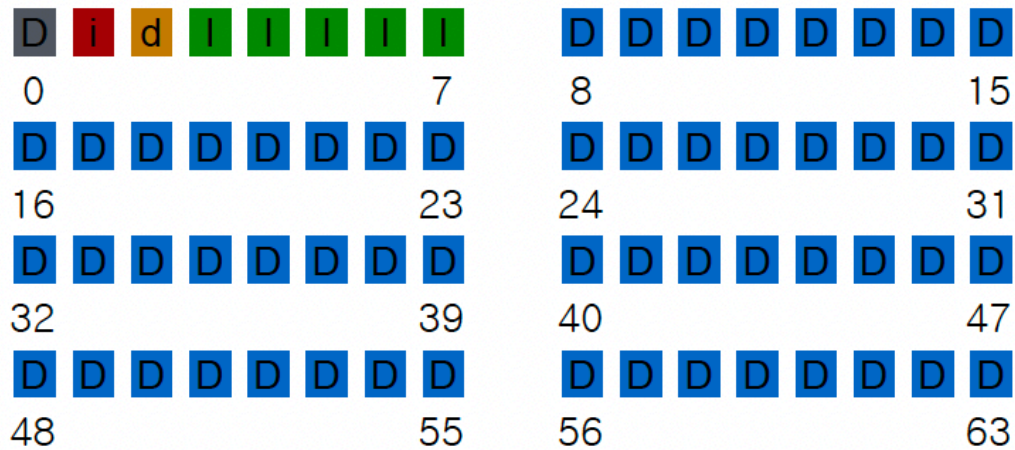


3번부터 7번 block : inode block

inode0 ~ inode15 : 첫번째 inode block

Allocation

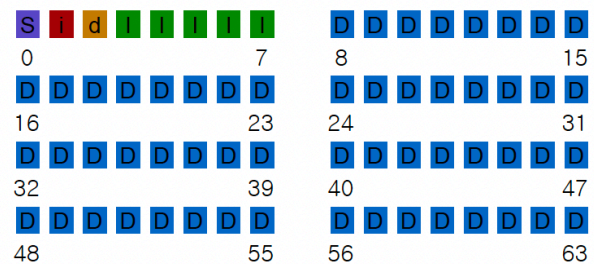
- How do we find free data blocks or free inodes?
 - Free list
 - Bitmaps → 이걸 사용



- i는 inode에 대한 bitmap
- d는 datablock에 대한 bitmap
 - d는 superblock

Superblock

- 0번지에 위치
- Need to know basic FS configuration metadata, like:
 - block size
 - # of inodes
 - Start location of inode table
 - etc
- Store this in superblock
- When mounting a file system, **OS will read the superblock first to initialize various information.**
- 깨지면 ㅈ됨



File Organization: The inode

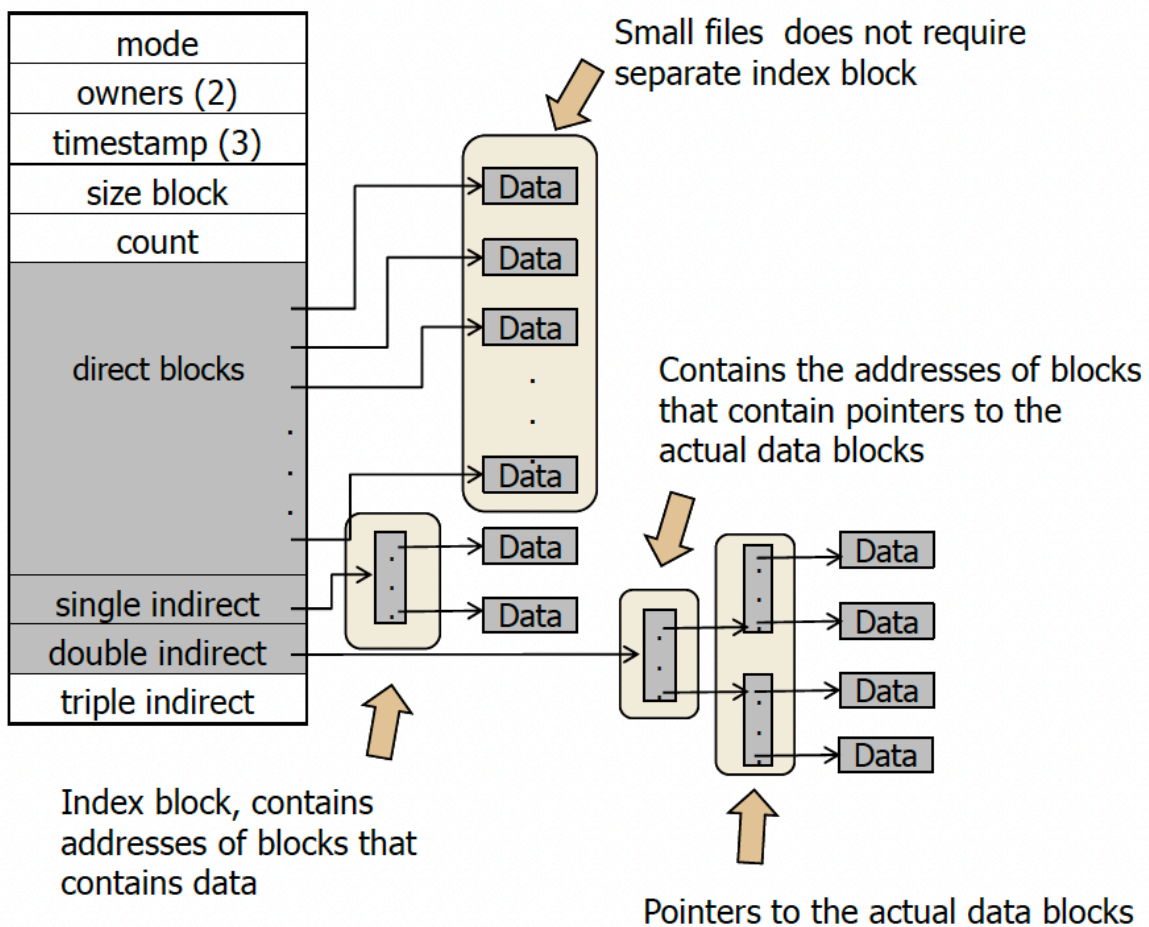
- Each inode is referred to by inode number.
 - by inode number, File system calculate where the inode is on the disk.

- Ex) inode number: 32
 - Calculate the offset into the inode region ($32 \times \text{sizeof}(\text{inode})$ (256 bytes) = 8192
 - Add start address of the inode table(12 KB) + inode region(8 KB) = 20 KB

The Inode table

					iblock 0				iblock 1				iblock 2				iblock 3				iblock 4			
					0	1	2	3	16	17	18	19	32	33	34	35	48	49	50	51	64	65	66	67
					4	5	6	7	20	21	22	23	36	37	38	39	52	53	54	55	68	69	70	71
					8	9	10	11	24	25	26	27	40	41	42	43	56	57	58	59	72	73	74	75
					12	13	14	15	28	29	30	31	44	45	46	47	60	61	62	63	76	77	78	79
0KB	4KB	8KB	12KB		16KB	20KB	24KB	28KB	32KB															

File Structure: Indexed Allocation



Directories

- File systems vary
- Common design:
 - Store directory entries in data blocks
 - Large directories just use multiple data blocks
 - Use bit in inode to distinguish directories from files
- Various formats could be used
 - Lists
 - b-trees

Simple Directory List Example

valid	name	inode
1	.	134
1	..	35
1	foo	80
1	bar	23

unlink("foo")

Directory Structure

한번 여러분들이 살펴 보세요

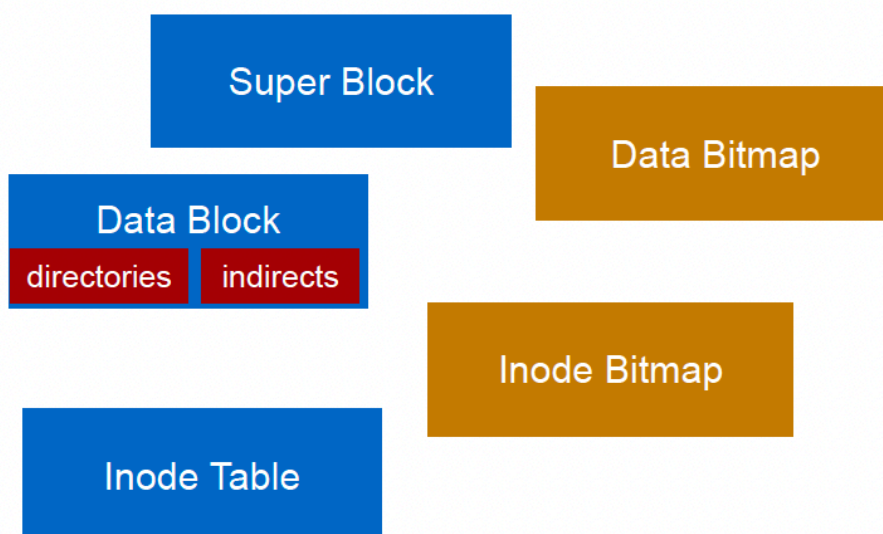
VSFS (Very Simple File System)

inum	reclen	strlen	name
5	4	2	.
2	4	3	..
12	4	4	foo
13	4	4	bar
24	8	7	foobar

EXT4

	inode	rec_len	file_type	name_len	name
0	21	12	1	2	· \0 \0 \0
12	22	12	2	2	· · \0 \0
24	53	16	5	2	h o m e 1 \0 \0 \0
40	67	28	3	2	u s r \0
52	0	16	7	1	o l d f i l e \0
68	34	12	4	2	s b i n

On-Disk Structures



Operations

create /foo/bar

data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data
		read			read	
			read			read
	read write					write
				read write		
			write			

- What needs to be read and written?

1. root inode read → root 디렉토리의 inode 읽기
2. root data read → root 디렉토리 내용 읽기 (foo가 어디 있나?)
3. foo inode read → /foo 디렉토리의 inode 읽기
4. foo data read → /foo 디렉토리 내용 읽기 (bar가 이미 있나 확인)
5. inode bitmap read → 새 파일(bar)을 위한 빈 inode 찾기
6. inode bitmap write → bar를 위한 inode 할당 표시
7. foo data write → /foo 디렉토리에 "bar" 엔트리 추가
8. bar inode read → 음... 이건 왜지? 🤔
9. bar inode write → bar 파일의 inode 정보 쓰기
10. foo inode write → /foo의 메타데이터 업데이트 (수정시간 등)

open /foo/bar

data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data
		read					
			read		read		
				read		read	

1. root inode read → root 디렉토리의 inode 읽기
2. root data read → root 디렉토리 내용 읽기 (foo가 어디 있나?)
3. foo inode read → /foo 디렉토리의 inode 읽기
4. foo data read → /foo 디렉토리 내용 읽기 (bar가 어디 있나?)
5. bar inode read → bar 파일의 inode 읽기

| write가 하나도 없어

write to /foo/bar

- assume file exists and has been opened

data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data
				read			
read write							
				write			write

1. bar inode read → bar 파일의 inode 읽기
 - 이미 open()으로 파일을 열었다고 가정!
 - file descriptor에 inode 번호가 있음
 - bar의 현재 크기, 데이터 블록 위치 등 확인
2. data bitmap read → 빈 data block 찾기

- 디스크에서 빈 블록 찾기
3. data bitmap write → data block 할당 표시
- 이 블록 이제 사용 중! 표시
4. bar data write → 실제 데이터 쓰기
- 실제 data를 data block에 쓰기
5. bar inode write → bar의 메타데이터 업데이트
- 파일 크기 증가, 수정 시간 업데이트, 새 data block 포인터 추가



왜 inode를 두 번 접근할까?

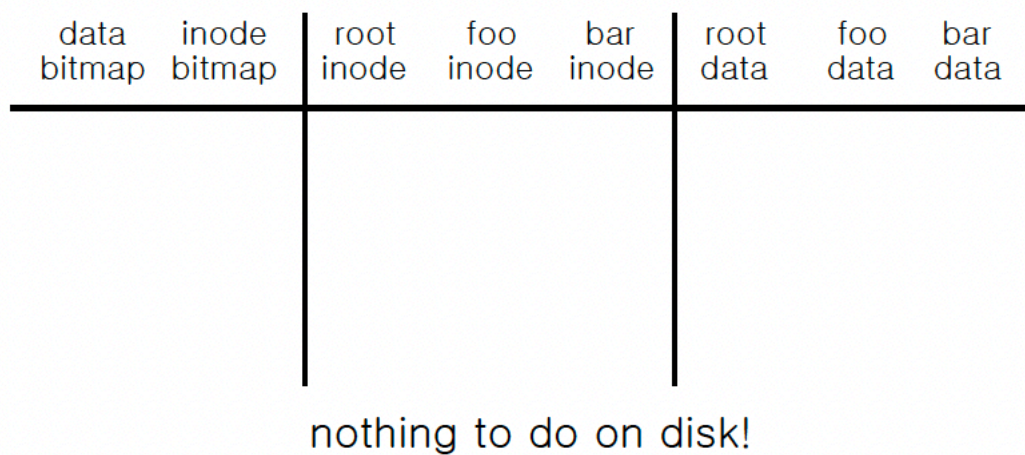
read-modify-write 패턴이야!

1. **Read**: 현재 상태 확인 (파일 크기, 어디에 쓸지)
2. **Modify**: 데이터 쓰기
3. **Write**: 변경된 메타데이터 반영

read /foo/bar - assume opened

data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data
				read			
				write			read

close /foo/bar



- file descriptor만 받는다?

Efficiency

- how can we avoid this excessive I/O for basic ops?
- cache for:
 - reads
 - write buffering

Write Buffering

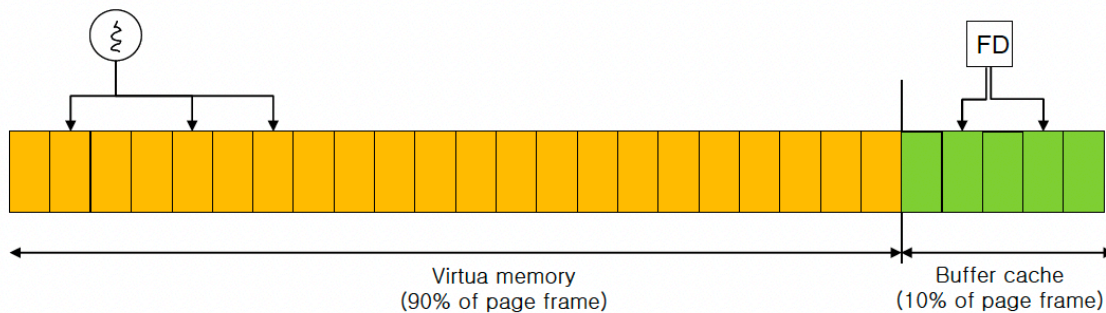
- Why does procrastination help?
- Overwrites, deletes, scheduling
- Shared structs (e.g., bitmaps+dirs) often overwritten.
- We decide: how much to buffer, how long to buffer... *tradeoffs*?

한번에 보내는게 효율적이다!

Caching and Buffering

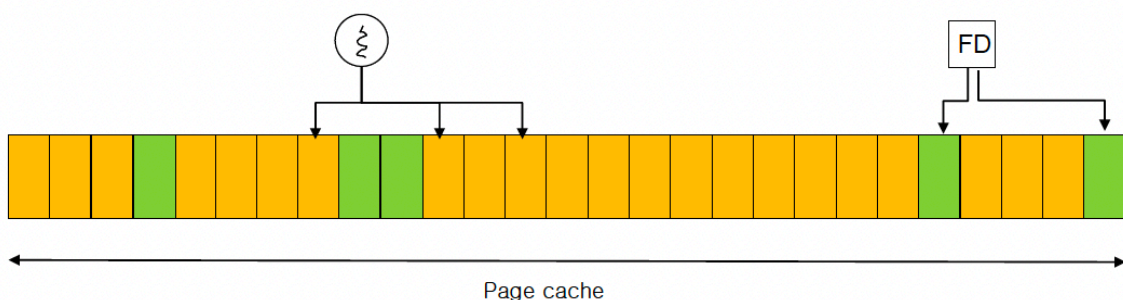
- Reading and writing can very IO intensive.
 - File open: two IO for each directory component and one read for the data.
- **Buffer Cache**
 - cache the disk blocks to reduce the IO.

- LRU replacement
- Static partitioning: 10% of DRAM, inefficient usage



• Page Cache

- Merge virtual memory and buffer cache
- A physical page frame can host either a page in the process address space or a file block.
 - Process uses page table to map a virtual page to a page frame.
 - A file IO uses "address_space"(Linux) to map a file block to a physical page frame.
- Dynamic partitioning



Summary

- We've described a very simple FS.
 - basic on-disk structures
 - the basic ops
- Future questions:

- how to allocate efficiently to obtain good performance from disk?
- how to handle crashes?